

CS4226 Internet Architecture

AY25/26 Sem 1. github.com/gerteck

0. Introduction

- Internet Architecture Prerequisites: Basic network terms including host, packet, protocol, throughput, bottleneck link, store-and-forward, and autonomous systems.
- Logical (five layers) and physical architecture (network of ASes). Pros and cons of packet switching versus circuit switching. Different components of end-to-end delay and relation to bandwidth, packet size, distance, propagation speed and queue size.
- CS3103:** Computer Networks Practice. Focus on practical, protocol level understanding (vertical view of protocol stack), with layers covered including data link, network and transport.
- CS4226:** Internet Architecture. Focus on theoretical, architectural and systemic (horizontal ecosystem view). Cross layer architectural design, such as peer-to-peer networks

1. Performance Metrics

Link Rate / Bandwidth / Capacity

- How many bits can be “pushed” onto a link per unit time. (Bandwidth)
- E.g. Cable: in Mbps, Fiber optics: $1 \sim 10$ Gbps, Ethernet: 3 Mbps to 100 Gbps
- Link rate related to transmission delay, or “putting furniture into the moving van”.

Throughput

- Throughput of a TCP/UDP flow. (End-to-End, includes propagation delay).
- How many bits communicated / pushed throughout entire routing path per unit time.

End-to-End Delay

- Components:** Processing delay + Queuing delay + Transmission delay + Propagation delay
- Processing (time taken by router/devices to process packet header), Queueing (time packet waits in buffer before it is transmitted, due to congestion), propagation delay (time taken for bit to travel from one end of link to the other.). Transmission delay is the time it takes for physical layer at source to send all bits of packet onto transmission medium.
- Variables:** Bandwidth, packet size, distance, propagation speed, queue size.
- Example: A packet of L bits sent over a link with rate R bits/sec takes.

Other Metrics

- Response Time:** Time taken from request to response, aka *round-trip time*, including server processing time etc. Traffic there and back may take different routing paths. Roughly 2x of end-to-end delay, but not exactly.
- Dropping Probability, Utilization**

1.1 Packet Switching: Store-and-Forward

- A packet of L bits sent on a link with rate R bps takes:

$$\text{Transmission delay} = \frac{L}{R} \quad \text{seconds to push out.}$$

- Store-and-forward:** entire packet must be at router before transmit to next link.
- E.g. $L = 7.5$ Mbits, $R = 1.5$ Mbps: Transmission delay per link = $\frac{L}{R} = 5$ sec. For 3 links: total delay = $\frac{3L}{R} = 15$ sec

1.2 Packet Switching: Statistical Multiplexing

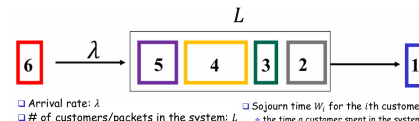
- Bandwidth shared on demand, no fixed pattern of packets. This is the origin of **queuing delay**. (Input rate $>$ output rate).
- Statistical multiplexing $\rightarrow$ efficient use of link capacity. Packet switching is more efficient to share resources among users. (Statistical: arrival of packets non-deterministic)
- Contrast with TDM (Time Division Multiplexing):
 - TDM: each host gets same slot in a revolving frame.
 - Statistical multiplexing: bandwidth allocated dynamically as needed.

Packet vs Circuit Switching

- Example: For Link capacity of 1 Mbps, where each user requires 100 kbps when active, active 10% of time.
 - Circuit switching** $\Rightarrow$ supports 10 users
 - Packet switching** $\Rightarrow$ with 35 users, probability > 10 active simultaneously is < 0.0004 .
- Pros of packet switching:** Great for bursty, asynchronous data, Resource sharing, No call setup required.
- Cons of packet switching:** Excessive congestion $\rightarrow$ packet delay and loss, requiring protocols for reliability and congestion control.

1.3 Single-Server Queueing Model (Abstract System)

- Model (to understand queueing delay): one server with infinite queue.
- Arrival rate: λ (packets per second).
- Service rate: depends on server capacity (e.g., 1.5 Mbps).
- Average queue length and waiting time linked by Little’s Law.



1.4 Little’s Law

$$L = \lambda W$$

- L : average number of customers in system
- λ : average arrival rate
- W : average sojourn time (time spent in system)

Examples:

- SoC admits 1500 students per year, has 6000 students enrolled at any time.

$$W = \frac{L}{\lambda} = \frac{6000}{1500} = 4 \text{ years}$$

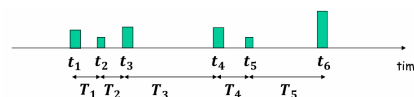
- If average queue length = 5 packets and arrival rate = 100/sec, then

$$W = \frac{L}{\lambda} = \frac{5}{100} = 0.05 \text{ sec}$$

2. Network Queueing & Models

2.1 Packet Flow: Arrival Pattern

- Arrival times: t_i ; Inter-arrival times: $T_i \equiv t_{i+1} - t_i$
- Model T_i s as independent, identically distributed (i.i.d.) random variable (r.v.) T



2.2 Probability Basics and Recap

- Random experiment: outcome cannot be determined in advance
- Sample space S : set of all outcomes; Event $E \subseteq S$: occurs if $s \in E$
- Probability function $P(E)$: $0 \leq P(E) \leq 1$, $P(S) = 1$

Random Variables (r.v.)

- Random variable X is a **mapping**: maps each outcome $s \in S$ to real value
- Distribution function $F(x) = P(X \leq x)$
- Continuous r.v.: exists pdf $f(x) = \frac{d}{dx} F(x)$
- Independence: $f_{X,Y}(x, y) = f_X(x)f_Y(y)$
- Expectation:

$$E[X] = \int_{-\infty}^{\infty} xf(x)dx \quad \text{or} \quad E[X] = \sum_x xP(X = x)$$

Exponential Distribution (Time between Intervals)

- $T \sim \text{Exp}(\lambda)$ with $\lambda > 0$: (note interarrival times cannot be negative).

$$f(x) = \begin{cases} \lambda e^{-\lambda x}, & x \geq 0 \\ 0, & x < 0 \end{cases}$$

- CDF $F(X)$:** probability that event occurs within time x , $P(T \leq x)$
- $F(x) = P(T \leq x) = \int_0^x \lambda e^{-\lambda u} du = 1 - e^{-\lambda x}$, $x \geq 0$
- Memoryless:** $P(T > s + t \mid T > s) = P(T > t)$
- Expectation:** (**E[T]**: Time till next event)

$$E[T] = \int_0^{\infty} x \lambda e^{-\lambda x} dx$$

Integration by parts:

$$E[T] = \left[-xe^{-\lambda x} \right]_0^{\infty} + \int_0^{\infty} e^{-\lambda x} dx = 0 + \frac{1}{\lambda} \Rightarrow E[T] = \frac{1}{\lambda}$$

- Physical meaning of λ :**

- λ is the arrival rate (arrivals per unit time), unit: time^{-1} (e.g., per second).
- On average, λ events occur per unit time, or the mean interarrival time is $1/\lambda$.

2.3 Modeling Packet Flow

Packet Arrival / Inter-arrival Times (Poisson Process, $T \sim \text{Exp}(\lambda)$)

- Inter-arrival times $T_i \sim \text{Exp}(\lambda)$, i.e. each inter-arrival time is an RV with rate λ .
- Arrival pattern:** Poisson process (# arrivals in an interval follow Poisson distribn.)
- Properties:** Memoryless $\Rightarrow$ starting time irrelevant, and Merging: $\text{Poisson}(\lambda_1)$ (combination) $\text{Poisson}(\lambda_2) \Rightarrow \text{Poisson}(\lambda_1 + \lambda_2)$

Service Time (Exponential / i.i.d.)

- Service time (processing time) of packet i : S_i , under fixed link rate.
- S_i are i.i.d. with mean $E[S] = 1/\mu$.
- Physical meaning of μ :** service rate, avg packets per unit time that can be handled
- Depends on link rate and packet size, as packet size may vary, hence, take average.

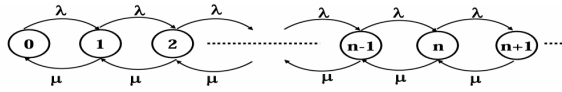
2.4 M/M/1 Model M (Markovian Poisson arrival), M (Markovian memoryless service time), 1 (# servers)



- Assumptions:
 - Single server, infinite queue size,
 - Poisson arrivals rate λ , Exponential service time rate μ
 - Independent arrivals and service, FIFO service discipline
- Utilization ρ** — (% time or probability of observation that server busy):

$$\rho = \frac{\lambda}{\mu}, \quad \text{need } \lambda < \mu \text{ for stability}$$

Birth-Death Process



- State Transition Diagram for M/M/1 System.
- System state $L(t)$ = (random) number of users in system.
- Transitions: $i \rightarrow i + 1$ at rate λ , $i \rightarrow i - 1$ at rate μ

Stationary Distribution (Long-term behavior of System)

- For a M/M/1 system, denote π_i as % time exactly i packets in system (ie. server + queue) / probability $P = \{L = i\}$ of random observation of i packets in system.
- $\pi_i = P(L = i) = (1 - \rho)\rho^i$, (follows a geometric distribution)
- $\pi_0 = 1 - \rho$ (utilization), and $\sum_i \pi_i = 1$

M/M/1 Results

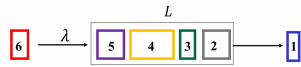
- Avg. # packets in system: $E[L] = \frac{\rho}{1-\rho}$
- Avg. sojourn time (system delay): $E[W] = \frac{1}{\mu - \lambda}$
- Avg. # packets in queue: $E[Q] = \frac{\rho^2}{1-\rho}$
- Avg. queueing delay of packets: $E[D] = E[W] - \frac{1}{\mu}$
- If $\lambda \geq \mu$, server keep up w. arrivals and queue grows without bound. In particular, $\lambda = \mu$ is unstable (graphically approaches vertical asymptote at $\lambda = \mu$):

$$E[L] = \frac{\rho}{1-\rho} \rightarrow \infty, \quad E[W] = \frac{1}{\mu - \lambda} \rightarrow \infty$$

Stability requires $\lambda < \mu$ (i.e., $\rho < 1$). ($\lambda = \mu$ insufficient bc. randomness).

2.5 Heavy Notation (Queueing Systems)

□ A general system:



- Deterministic system

□ A special system:

M/M/1 Model



- Randomness is introduced
- Require different form of notation.

□ Times:

- time t and arrival time t_i
- inter-arrival time $T_i \triangleq t_{i+1} - t_i$
- sojourn time $W_i \triangleq t_i^d - t_i$,
- mean sojourn time
 - as a deterministic limit: $W \triangleq \lim_{n \rightarrow \infty} \frac{1}{n} \sum_{i=1}^n W_i$
 - as a random variable: $E[W]$
- in a queueing system: $W_i = D_i + S_i$
 - mean queueing delay $E[D]$
 - mean service time $E[S]$

□ Numbers:

- # of packets/jobs/users $L(t)$
- average/mean # of packets in the system
 - as a deterministic limit: $L \triangleq \lim_{t \rightarrow \infty} \frac{1}{t} \int_0^t L(s) ds$
 - as a random variable: $E[L]$
- in an M/M/1 queueing system: $E[L] = E[Q] + E[X]$
 - mean # of packets in the queue $E[Q]$
 - mean # of packets in the server $E[X]$
- $E[X] = P(X=0) \cdot 0 + P(X=1) \cdot 1 = P(X=1)$
- $P(X=1) = P(L-Q=1) = P(L>0) = 1 - \pi_0 = \rho$

□ Rates:

- arrival rate λ
 - as a deterministic limit: $\lambda \triangleq \lim_{t \rightarrow \infty} \frac{N(t)}{t}$
 - as a function of random variable: $\lambda \triangleq \frac{1}{E[T]}$
- service rate μ
 - defined in a queueing system as: $\mu \triangleq \frac{1}{E[S]}$

2.6 Effective Bandwidth

- Physical link capacity: theoretical hardware processing limit (E.g. 10Mbps cable)
- Effective bandwidth:** actual throughput achievable, given service quality constraints. E.g. for μ capacity of M/M/1 link:
- (utilization constraint of $\rho \leq \hat{\rho}$, low delay (delay bounded by $D \leq \hat{D}$, where $E[D] = \frac{\rho}{\lambda - \mu}$, low loss etc) .

2.7 Statistical Multiplexing vs Time Division Multiplexing

- Time Division Multiplexing (TDM):** Each of k users given fixed fraction of link capacity μ/k , regardless of idleness (wasted capacity).
- With k streams, $\rho = \frac{\lambda}{\mu} = \frac{\lambda/k}{\mu/k} = \rho$ (same server utilization), but avg. sojourn time ($E[W]$) increased by k times under TDM. Queue length ($E[Q]$) and total users ($E[L]$) also k times larger
- Statistical Multiplexing:** Multiple traffic streams share the common link dynamically, each flow sends when has data, link schedules packets one by one from queue, bandwidth allocated statistically (random arrivals), not deterministically. (e.g. M/M/1 FIFO queue model)
- Statistical better than TDM. (same utilization, average less delay, shorter queue).

2.8 Variants of Queueing Models

- Multiple servers: M/M/2, Limited queue size: M/M/1/K
- Different service: M/M/1/LIFO, General distributions: M/G/1, G/M/1, G/G/1

2.9 Burke's Theorem

- For M/M/1 queue in steady state with arrival rate λ and service rate $\mu > \lambda$
- Departure** process of form Poisson process of rate (λ)
- Independence:** System size at time t independent of past departure times
- Implication is that output of one M/M/1 queue can be treated as Poisson input for the next queue, simplifying analysis of multi-hop queueing systems.

2.10 Tandem Queues

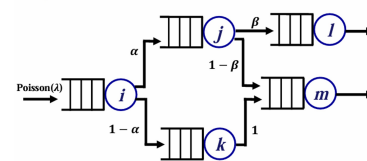
- Tandem Queue:** System where jobs pass through multiple servers in sequence, one after another. Each server i modelled as an M/M/1 queue with own service rate, and utilization of server i : $\rho_i = \lambda/\mu_i$
- Joint probability of no. of jobs in each subsystem (independent):

$$P(L_1 = j, L_2 = k) = (1 - \rho_1)\rho_1^j (1 - \rho_2)\rho_2^k$$

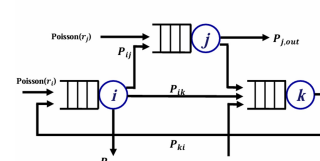
2.11 Jackson Network of Queues

- Firstly, a **Acyclic Network with Probabilistic Routing** is where jobs arrive at a network node, and choose next node randomly. Poisson property preserved, departures remain independent Poisson processes (Poisson splitting)
- Jackson Network:** Network of queues (e.g. M/M/1) with probabilistic routing between servers P_{ij} , and can be acyclic or have feedback (cyclic).

Acyclic Network with Probabilistic Routing



Jackson Network



- Solve Jackson Network,** Burke's theorem merge & split of Poisson still apply.
- Two-step procedure:** determine utilization ρ_i for each server, and product form solution.
- Effective arrival rate:** λ_i at server i : $\lambda_i = r_i + \sum_{j=1}^n \lambda_j P_{ji}$ (r is outside arrival)
- In matrix form: $\lambda = r + \lambda P \Rightarrow \lambda = r(I - P)^{-1}$
- Product form solution: $P(L_1 = l_1, \dots, L_n = l_n) = \prod_{i=1}^n (1 - \rho_i)\rho_i^{l_i}$

Example Jackson Network: Web Server

CPU and I/O modeled as two servers

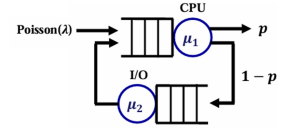
Effective arrival rates:

$$\lambda_1 = \lambda + \lambda_2, \quad \lambda_2 = (1 - p)\lambda_1$$

Average jobs in each:

$$E[L_i] = \frac{\rho_i}{1 - \rho_i}, \quad \rho_i = \frac{\lambda_i}{\mu_i}$$

Total jobs: $E[L] = E[L_1] + E[L_2]$



3. Resource Allocation

3.1 Queues - Delay (sojourn $E[W]$), capacity (service μ), arrival (λ)

- Average waiting (sojourn) time** in an M/M/1 queue is $E[W] = \frac{1}{\mu - \lambda}$, where μ is the service rate (i.e. called capacity) while λ is arrival rate. More capacity (higher service rate) means higher throughput, and lower delay.
- Max throughput** λ for a given delay requirement $E[W]$ would be $\lambda = \mu - \frac{1}{E[W]}$, determined by service rate.
- Physical Interpretation of λ :** Here, because we want to guarantee some delay requirement, for a given service rate / capacity, the total throughput has to be bounded. 'Arrival rate' λ (throughput) here is instead admission control, where system limits the incoming rate of incoming flow to guarantee latency. Not the senders decision (e.g.UDP), but instead, it is a resource allocation problem.

3.2 Motivation of Resource Allocation

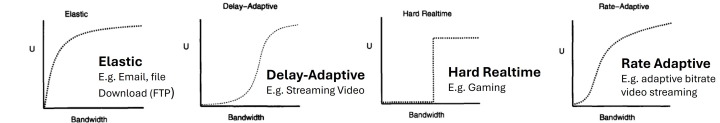
Internet provides **best-effort** service. Network resources (bandwidth, buffer space, processing time) are limited. Hence, problem to allocate resources fairly and efficiently across competing flows. Tradeoff between **efficiency** (maximize utilization) and **fairness** (avoid starvation).

Resource Allocation in Networks

General Utility Perspective

Traffic Classes in Networking
(different utility functions)

User's happiness U_i as a function of performance metrics, e.g. delay or throughput



Metrics: *Throughput:* average rate of successful packet delivery, *Delay:* time from packet arrival to departure, *Fairness:* how equally bandwidth is shared among flows, *Loss rate:* fraction of packets dropped due to congestion.

3.3 Max-Min Fairness

Idea of fairness where low demand users get what they need, large demand users share the rest equally (fair allocation).

- Max-min fair:** A feasible allocation max-min fair iff cannot increase anyone's rate without decreasing someone else's rate that is smaller or equal.
- Also exists, typically unique. Ensures no flow is starved.
- Flow's Bottleneck Resource:** A resource r is a bottleneck for flow i if r is saturated (fully used) and flow i has max rate among all flows using resource r (both conditions).
- Note: **weighted (ϕ) flow:** For flow bottleneck resource for weighted max-min, flow i has max (**normalized**) rate among all flows. (Not max absolute rate).
- Theorem:** With infinite demand, a max-min fair allocation iff every flow i has a bottleneck resource r .

Water-Filling Algorithm

The water-filling algorithm is a practical way to compute max-min fair allocations.

- **Flow** = bucket, **water** = system resources

Each flow’s demand is “bucket” with volume d_i and system resources as “water” poured until capacity is exhausted.

- **Step 1:** Represent each demand d_i as a bucket (capacity limit).
- **Step 2:** Start filling all buckets equally (same rate for each flow).
- **Step 3:** When a bucket demand full (or the flow’s bottleneck link saturates - flow hits bottleneck), stop filling; continue distributing remaining capacity among other unfinished buckets. (I.e. stop partial allocations for certain flows that have been constrained by their bottleneck link).
- **Step 4:** Continue until all capacity is allocated or all demands are satisfied.

Weighted Max-Min Fairness

Extension of the water-filling algorithm with weights ϕ_i . Weighted fairness generalizes the idea to assign priorities via weights.

- Buckets now have different *widths* proportional to ϕ_i (instead of equal).
- Each flow’s rate grows proportionally to its weight until either demand is satisfied or capacity is exhausted.
- No user receives more than their demand. Remaining resources are split proportionally.

Implementation of Water-Filling Algorithm

We do not know in advance which resource will be the next bottleneck, or at what “water level” capacity will be fully utilized.

1. Start by increasing all flows equally (or proportionally if weighted).
2. For each resource, check its capacity constraint $\sum_{i \in R} x_i \leq C_R$.
3. The first resource that saturates defines the next bottleneck; flows using this resource are then capped.
4. Remove the capped flows from further consideration and continue filling the others.

This can be executed locally:

- Each router enforces per-flow constraints on its outgoing link.
- Flows naturally converge to a max-min fair allocation since no flow can exceed its bottleneck capacity.

Implementation of Max-Min Fairness

In practice, implementing max-min fairness in the Internet is challenging due to the lack of centralized control. Routers must make local scheduling decisions in a distributed fashion.

- **Challenge:** No single global controller; each router only sees its local queues.
- **Classical solutions:**
 - GPS (Generalized Processor Sharing): theoretical ideal where packets are served bit-by-bit in perfect proportion to their weights.
 - WFQ (Weighted Fair Queueing): practical approximation of GPS, where each packet is stamped with a virtual finish time and routers serve packets in increasing finish-time order.
- **Modern solution:** SDN (Software Defined Networking) provides centralized or global control of flows, enabling explicit allocation strategies across the network.

4. Software Defined Networking (Design Principles)

Software Defined Networking (SDN) is a new approach to implement and manage networks based on fundamental principles that impact both network design and the pace of innovation. Traditional networking has been successful, but has significant limitations:

- Closed and proprietary equipment → little room for innovation.
- Over-specified protocols → slow standardization and evolution.
- Complex, hard-to-manage systems → high operational costs and frequent outages.

SDN proposes to solve these challenges by separating the **control plane** and the **data plane**, centralizing control, and enabling programmability.

4.1 Why SDN? Problems with Traditional Networking

1. **Tremendous success, but only at the edge:** The Internet evolved from research to global infrastructure, great innovation at the application layer (7) (web, VoIP, P2P, etc.), but very little innovation inside the network.
2. **Inside the network:** Closed equipment where hardware and software bundled vendor specific interfaces limit flexibility. Over specified, Protocol standardization is slow → decades to evolve routing protocols. Innovation only up to equipment vendors, so slow.
3. **Networks are hard to manage:** Operating costs exceed hardware costs. Human error largest cause of outages. Network software is massive and buggy (e.g., routers with 20M+ lines of code).
4. **Networking lags behind other systems:** (Compute, Storage, Networks) Computation and storage have been virtualized, enabling rapid innovation. Networking, however, is still highly manual and primitive in management. - New control requirements just lead to greater complexity (isolation, packet processing, payload analysis).

Networking as a Discipline

Unlike OS or DB systems, networking has not converged around basic principles or abstractions. Instead, it is taught as a “*bag of protocols*”. SDN tries to extract simplicity using modular abstractions, much like programming evolved from machine code to higher-level languages with abstracted functions etc.

The Network Control Problem

The traditional (complex) control problem is:

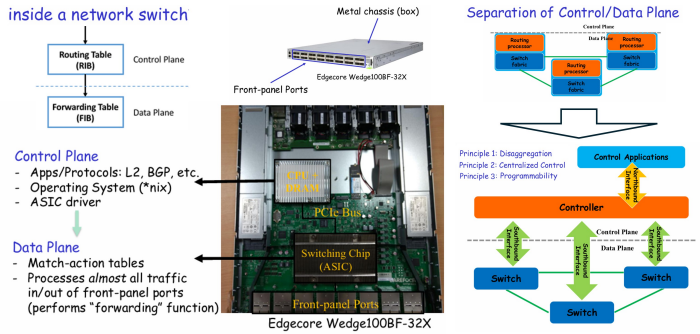
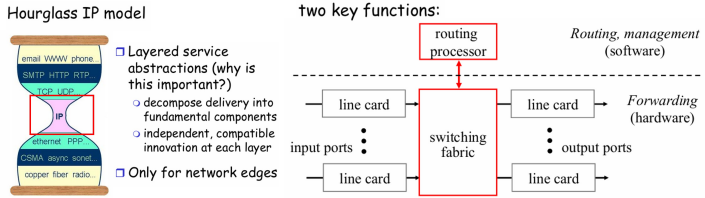
- Compute forwarding tables for each device.
- Operate without reliable communication guarantees.
- Operate within given distributed routing protocols (e.g. OSPF, RIP, BGP).

This distributed complexity makes it difficult to innovate or manage.

4.2 Control Plane vs. Data Plane

Two key functions of router and switch at the core: Routing algorithms/protocol vs. Forwarding datagrams.

- **Control Plane:** Decides how packets should be forwarded (routing, traffic engineering, firewalling). Operates at slower time scales (per-event basis).
- **Data Plane:** Forwards packets based on control plane instructions. Operates at very fast timescales (per-packet basis).



Principle 1: Disaggregation

Traditional networking devices bundle control and data plane tightly. **Disaggregation** separates them via open interfaces:

- Commodity switches handle packet forwarding (data plane).
- Control plane software can run independently, possibly from different vendors.
- **Forwarding Abstraction:** need to define, as a general-purpose way for control plane to tell data plane to forward pkts in specific way.
- E.g.: *OpenFlow* defines **flow rules** (match-action pairs) as forwarding abstraction.

Principle 2: Centralized Control

Instead of every router running its own control software:

- A logically centralized controller has a global network view.
- Simplifies management compared to distributed routing (contrast: OSPF’s 244 pages vs Dijkstra’s 4 pages).
- Physically centralized controllers may have fault-tolerance issues, hence the need for logically centralized but physically distributed implementations.

Principle 3: Programmability

- Control shift: vendors → operators → users.
- Networks programmable via high-level policies. Debug, verification easier.
- Enables rapid innovation and new services (e.g., network slicing in 5G, dynamic security policies in data centers).

4.3 Benefits of SDN

1. **Easier management:** Network policies are written at a high level; low-level configurations are handled by the controller.
2. **Faster innovation:** Network control software evolves independently from hardware. Disaggregation shifts control from vendors to operators or open source that build networks to meet their users needs.
3. **Cheaper infrastructure:** Commodity (standardized) forwarding devices replace expensive proprietary hardware.
4. **New use case:** Data center virtualization, traffic engineering, dynamic QoS...

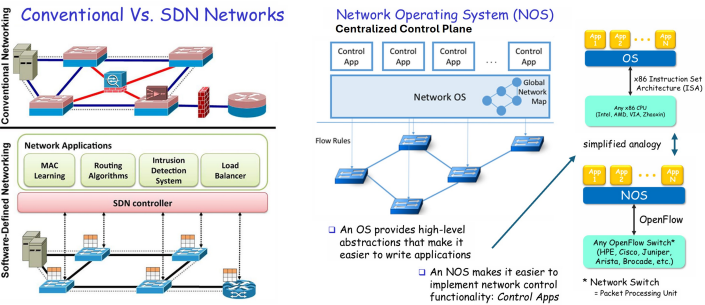
4.4 Abstraction Concepts

These 3 principles modularize the **network control problem** into something similar to OS computation abstractions:

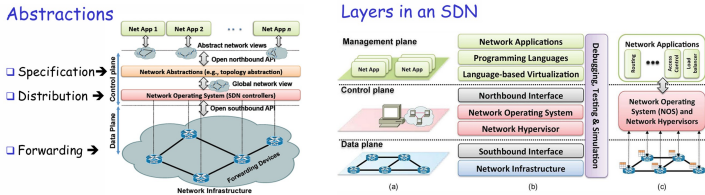
- **Specification abstraction:** Control applications specify desired behavior without worrying about implementation.
- **Distribution abstraction:** Shields applications from distributed system complexity, presenting a logically centralized view..
- **Forwarding abstraction:** Provides a generic interface to define how packets should be forwarded (e.g., OpenFlow).

SDN makes networks easier to program, easier to manage, and cheaper to build, while enabling faster innovation and richer functionality.

Network Operating System



SDN: An abstraction for Networking



4.5 Recap: Core Principles of SDN

SDN is built on three core design principles that aim to simplify and modularize the control of large-scale networks:

- 1. **Separation of Control Plane and Data Plane** – Network devices focus only on packet forwarding (data plane), while decision-making logic (control plane) is moved to a separate controller.
- 2. **Logically Centralized Control** – A centralized controller maintains a global view of the entire network, enabling coherent and optimized control decisions.
- 3. **Programmability** – The controller exposes APIs that allow applications and operators to program the network dynamically, enabling automation, customization, and rapid innovation.

Each principle leverages set of **abstractions** to network control complexity:

- **Specification abstraction** – Allows applications to express *what* behavior is desired, without specifying *how* it is implemented.
- **Distribution abstraction** – Presents a logically centralized view, hiding the distributed nature of devices and controllers.
- **Forwarding abstraction** – Defines how the control plane communicates forwarding behavior to the data plane (e.g., OpenFlow rules).

5. SDN - Use Cases, OpenFlow, ONOS

5.1 Users and Applications of SDN

Who uses SDN today?

- **Cloud providers:** Google, Facebook, Microsoft - leverage SDN to manage large-scale data centers, WANs efficiently. Also release open-source components (e.g., ONOS, OpenFlow) to drive ecosystem adoption.
- **Network operators:** Comcast, AT&T, NTT - adopt SDN gradually for traffic engineering and operational flexibility.
- **Enterprises:** Universities (first to adopt - from research) and private companies - use SDN in managed edge networks and via commercial SDN products.

5.2 Varied Use Cases of SDN

- 1. **Network Virtualization:** Virtualization technology for networks limited to VLANs, VPNs, only virtualizing address space, unlike compute VMs, containers virtualization. Modern clouds want to be able to:
 - Programmatically create, manage, and delete virtual networks via APIs.
 - Disaggregate physical networks into multiple virtual ones with isolated address spaces.
 - Treat each virtual network as an independent logical network, managed by SDN controllers.
- SDN enables this: via centralized control and programmable APIs that create, tear down virtual networks dynamically, w/o manual configuration. Each virtual network can define own policies, routing, topology, like how VMs abstract physical servers.

- 2. **Switching Fabrics in Data Centers:** Cloud datacenters want lower costs via commodity hardware, high scalability, fault tolerance, simplified multi-path connectivity for large server clusters. SDN use case: **Leaf-spine topology** :

- Each leaf switch (top of rack ToR switch) connects to all spine switches.
- Any two servers two hops apart (within same rack) or four hops (across racks).
- Provides uniform latency and multiple equal-cost paths for load balancing.

With SDN, central controller can monitor traffic pattern, do dynamic load balancing to distribute workload, installs flow rules to dynamically manage traffic paths across spine-leaf fabric, optimize utilization and fault recovery.

- 3. **Traffic Engineering (TE):** Wide-area links (e.g., btwn Google’s global data centers in B4 (private WAN), or Microsoft’s SWAN) often experience variable load and congestion. SDN enables controller to:

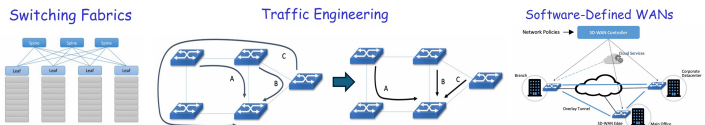
- Collect real-time link utilization data and recompute optimal routes centrally.
- Reconfigure flows dynamically to balance load across available paths.

SDN allows defining flows with different delay and availability requirements (different traffic classes, i.e. Elastic, Delay-adaptive, Rate-adaptive, Hard real-time traffic.)

- 4. **Software-Defined WAN (SD-WAN):** applies SDN principles to connect geographically distributed sites securely over public or private WAN links.

- Dynamic routing based on performance (latency, loss, cost), Centralized policy control and monitoring, Improved reliability via automatic failover.

Companies may serve customers in different countries, needing to adhere to local data laws. SDN enables enterprises to replace expensive MPLS circuits w. cheaper broadband or 5G links while maintaining performance guarantees.



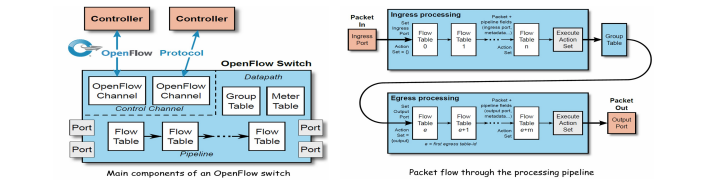
Summary of SDN Use Cases

- **Killer applications of SDN:** Network virtualization, data center networking, cloud computing. (i.e. software program or feature so valuable/popular, drives adoption of larger technology, platform, or device)
- **Emerging applications:** Last-mile access networks, software-defined Internet exchanges (SDX), wireless management, network security, monitoring.

5.3 OpenFlow API (Forwarding Abstraction)

Standard interface btw. control plane (SDN controller) and data plane (switch). **OpenFlow:** foundational protocol, controller communication (how to process packets) w. forwarding devices (switches, routers).

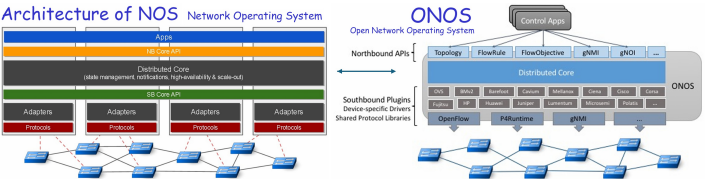
- **Open southbound API:** Enables ext software control over device forwardg tables.
- **OpenFlow switch:** Implements packet-matching and action-execution pipelines.
- Each switch has set of **Flow Tables**. Flow Entry has **Match fields** (conditions on header/pipeline fields (e.g., IP, TCP port)), **Priority** (on multiple entry matches), **Instruction set:** (actions to perform - forward, drop, or send to controller).
- A packet is processed through one or more flow tables. Matching rules accumulate an **action set**, which executes once processing is complete.
- Default “table-miss” entry defines for unmatched packets (e.g. send to controller)



5.4 Open Network Operating System (Specification Abstraction)

ONOS: distributed system, open-source SDN controller developed by the Open Networking Foundation (ONF).

- **Specification abstraction (northbound APIs):** Apps can query network state, define intents (high-level connectivity goals), or request packet interception w/o implementation details.
- **Distribution abstraction (core services):** Presents logically centralized view by syncing state across distributed controller cluster.
- **Forwarding abstraction (southbound interface):** Communicates with devices using protocols such as OpenFlow or NETCONF.



ONOS Architecture: *Northbound Interface (NBI)*, APIs used by apps to query topology, intercept packets, control forwarding behavior. *Distributed Core* maintains network state using a scalable key/value store (*Atomix*), ensuring high availability and consistency. *Southbound Interface (SBI)*, plugin-based interface for different device protocols and drivers.

ONOS programming constructs: *Intent:* High-level general policy (e.g. connect Host A - Host B w. 10 Mbps guarantee). *Flow Objective:* Mid level, device agnostic rule translating intent to device-specific logic. *Flow Rule:* Low-level hardware rule installed in device tables (e.g., OpenFlow entries).

ONOS (Network Operating System) as a horizontally scalable cloud application: has set of loosely coupled subsystems, often associated with micro-service architecture, coordinating distributed network devices, like how traditional OS manages hardware resources.

5.5 SDN Abstraction Summary

SDN abstracts the network into three layers: **Apps** that express high-level policies via APIs, **Control Plane (Controller)** that implements logic, optimization, and state management and **Data Plane (Devices)** that executes rules defined by controller. SDN transforms rigid networks to open programmable infrastructures.

6. SDN - P4 Programming Language

6.1 What is P4

- **P4** is a programming language for controlling packet forwarding planes in networking devices, such as routers and switches.
- **Domain-specific** language with constructs optimized for network data forwarding. Open source. P4: Programming Protocol-Independent Packet Processors. (i.e. domain is network packet processing)

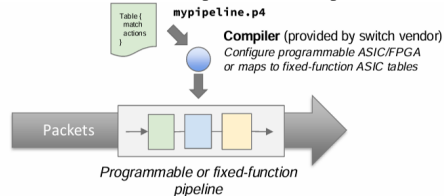
Why P4

- **Limitation of traditional switches:** Fixed-function ASICs (app-specific integrated circuits) can only process a pre-defined set of protocols (e.g., Ethernet, IPv4, TCP). Any new header format or protocol change requires new hardware. Slow innovation cycle, standardization, ASIC redesign take years.
- **Limitations of OpenFlow:** OpenFlow gives standard API for programming behavior of switches (remote control), but only supports fixed set of header fields (e.g. TCP, VLAN). New protocol (e.g. QUIC, custom headers) means must update OpenFlow standard and hardware vendors.
- Control over flow table rules, but none over packet type, packet parsing, processing inside switch.

Goal of P4:

- **Program the data plane itself**, and not just configure flow rules.
- Define new packet header formats, parsing logic, match-action tables, and actions.
- Enable switches to process any kinds of packets, support protocol evolution with need for hardware redesign, achieving protocol independence (e.g. no assumption on IPv4, TCP, etc.)

P4 defines the data plane pipeline, can describe fast pipelines (e.g. ASIC, FPGA) and slower ones (e.g. SW switch). Good for both programmable, fixed-function switches. Defines contract btw. control plane and data plane for runtime control.



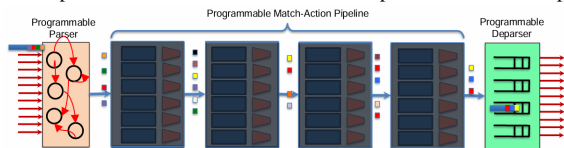
6.2 P4 Logical Pipeline: Programmable Match–Action Pipeline

Every P4-based switch implements a **logical pipeline** of match–action tables that process packets at line rate.

Input Packet → Parser → Ingress Tables → Traffic Manager
Traffic Manager → Egress Tables → Deparser → Output Packet

- **Parser:** Extracts header fields (Ethernet, IP, etc.) using a finite-state machine.
- **Ingress Pipeline:** Match–action stages that determine forwarding or modification.
- **Traffic Manager (TM):** Handles queuing, buffering, and replication (typically fixed-function but configurable).
- **Egress Pipeline:** Processes packets before transmission (e.g., tagging, metering).
- **Deparser:** Reassembles headers and payloads into a valid packet.

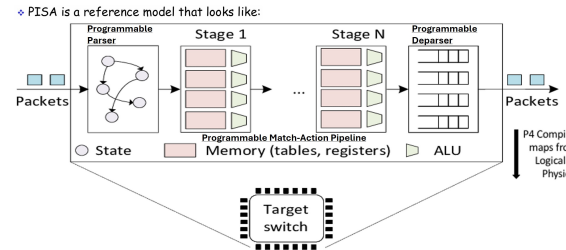
Parser “reads” packet, tables “decide” what to do, deparser “writes” final packet.



6.3 Formalization: (PISA) Protocol-Independent Switch Architecture

PISA is a **logical reference model**, defines the standard logical architecture that programmable data planes follow.

- Provides a vendor-neutral abstraction for packet processing, defining programmable blocks: *parser*, *ingress*, *egress*, and *deparser*. It allows the same P4 program to be compiled for different hardware targets.
- **Benefits:** Hardware independence, program can run on different vendor devices. Clean abstraction of logical packet processing from physical implementation. Faster innovation, new protocols, header fields can be supported instantly.



6.4 P4 Architecture and Targets

Mapping from Logical to Physical: P4 still needs to account for different chips implementing different physical pipelines. Hence, allows for vendor-specific logical pipelines, through different architectures, evolve from P4₁₄ to P4₁₆. P4 separates concepts of **architecture** and **target**:

- **Architecture:** Logical model / interface defining programmable components.
- **Target:** Specific hardware implementation running the compiled P4 program.
- Concept of an architecture defines what target hardware supports. P4 compiler translates logical design from P4 source to what fits physical chip architecture targeted.

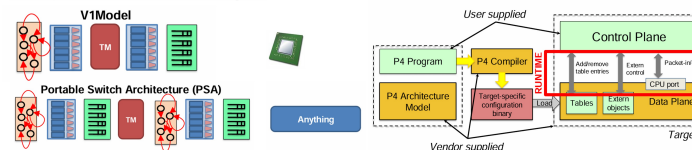
Common Architectures:

- **V1Model:** Basic architecture used in software switches like bmv2.
- **PSA (Portable Switch Architecture):** A standardized, general-purpose model.
- **PNA (Programmable NIC Architecture):** For smart network interface cards.

Common Targets:

- bmv2 (Behavioral Model v2) – software switch for development and testing.
- Intel Barefoot Tofino ASIC – high-performance programmable hardware switch.

Example Architectures and Targets



6.5 Writing a P4 Program: A Practical Look

A P4 program for the V1Model architecture follows a well-defined structure that corresponds to the programmable match–action pipeline.

- **Includes & Definitions:** The program begins by including the core P4 libraries and the architecture model file (e.g., v1model.p4). Also defines *Packet headers* (e.g. Ethernet and IPv4, and *Metadata*, both standard (e.g., ingress.port) and user-defined.
- **Parser:** Implements a finite-state machine that reads incoming packets and extracts relevant header fields to identify the packet type.

- **Control Blocks (Ingress/Egress):** The core logic of packet processing is defined here as a series of match–action tables. Each table specifies:
 - **Key:** Field(s) to match on (e.g., ipv4.dstAddr using longest-prefix match).
 - **Actions:** Possible operations to perform (e.g., ipv4.forward, drop).
 - **Size:** Maximum number of table entries allowed.

Actions themselves are defined as procedures that manipulate headers or metadata, such as setting egress.port or decrementing the TTL.

- **Deparser:** Reassemble modified headers, payload into packet for transmission.
- **Instantiation:** Last, all components (*parser*, *ingress*, *egress*, *deparser*) wired together to form full logical pipeline.

Overall, P4 program describes logical behavior of packet processing from parsing headers, making forwarding decisions, to rewriting and emitting packets, mapped to the architecture’s programmable pipeline.

6.6 P4Runtime and Control–Data Plane Interaction

- **Data Plane:** Executes the P4 program, processes each packet according to installed table entries.
- **Control Plane:** Runs externally (e.g., ONOS, Ryu, or custom controller). It inserts, updates, and deletes entries in match–action tables. Uses the **P4Runtime API** to communicate with the data plane.
- **P4Runtime:** A standard, gRPC-based API between the control plane and P4 devices. It allows runtime configuration, counter reading, and table management. Makes P4 switches vendor-independent and interoperable.
- **Example:** The control plane can install a routing rule (destination prefix → output port) in a P4 switch using P4Runtime.

6.7 Real-World Constraints and Considerations

- **Hardware limits:** P4 targets have finite memory and pipeline stages.
- **Limited arithmetic:** Only simple operations (add, shift, subtract) are allowed to maintain line-speed.
- **Offload to control plane:** Complex computations can be sent to CPU if needed.
- **Compiler mapping:** P4 compilers optimize logic based on the hardware’s capacity.

6.8 Example: IPv4 Router in P4

- **Parser:** Extract Ethernet and IPv4 headers.
- **Ingress Table:** Match destination IP (LPM) → next-hop MAC + output port.
- **Action:** Rewrite MAC, decrement TTL, forward.
- **Control Plane:** Installs and updates routing table entries at runtime.
- **Result:** A fully functional IPv4 router implemented purely in P4 — extendable to other protocols (e.g., IPv6, MPLS) without hardware change.

6.10 P4 Summary and Key Takeaways

- P4 introduces true **data plane programmability**.
- PISA provides a universal, logical model for switch pipelines.
- **P4Runtime** bridges control and data planes via a vendor-neutral API.
- Tools like bmv2 and Mininet enable software experimentation.
- **Mindset:** Move from configuring fixed hardware to programming network logic directly. Enables networks to evolve as fast as software.

7. Internet Interconnection

7.0 The Internet

The Internet is fundamentally a **network of networks**. These networks are organized, interconnected, and governed by both technical and business relationships. Focus on three key components:

- 1. **Autonomous Systems (AS):** the basic unit of Internet organization.
- 2. **Internet Peering:** bilateral interconnection through business relationships.
- 3. **Internet Exchange Points (IXP):** multilateral interconnection for efficient traffic exchange.

Internet interconnection defines how ASes cooperate & compete to maintain global connectivity, shaped by technical routing policies and commercial incentives. **Inter-domain routing protocols**, particularly the Border Gateway Protocol (BGP), enforces these policies across the global Internet.

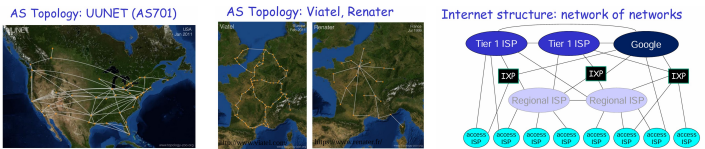
7.1 Autonomous Systems (AS)

- **Autonomous System (AS)** is a collection of routers operated under a single administrative domain that shares a consistent routing policy.
- Each AS is uniquely identified by an *Autonomous System Number (ASN)*, which is allocated by the **Internet Assigned Numbers Authority (IANA)** through one of the five **Regional Internet Registries (RIRs)**.
- Public AS databases include `db-ip.com/asn` and `peeringdb.com`.
- An organization, such as an ISP, corporation, or university, may manage multiple ASes. Each AS determines which other networks to interconnect with, which routing policies to apply, and how to manage internal routing. ASes use a common internal routing protocol and appear as single entities in inter-domain routing.
- **Logical Unit:** AS is like a unit of network organization, with defined routing policy and networks.

7.2 Internet Hierarchy and Structure

The Internet has a loosely hierarchical structure:

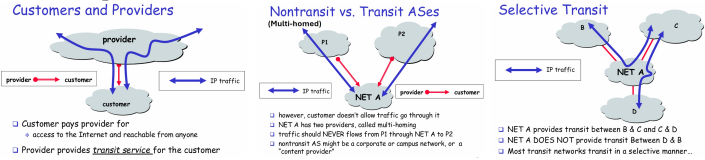
- **Tier-1 ISPs:** At the **core/top**, few large and globally connected. (e.g., AT&T, Level 3, Sprint, NTT). Peer with one another in full mesh, do not pay for transit.
- **Tier-2 ISPs** have national or regional coverage, purchasing transit from Tier-1 ISPs while also peering selectively.
- **Stub ASes**, such as universities or corporate networks, connect to one or more upstream providers and do not provide transit for others.



Internet Service Providers and ASes:

- **ISP:** provides internet access, owns and operates one or more ASes.
- Content providers like Google have also developed private global networks that often bypass traditional Tier-1 ISPs via direct peering and IXPs. The hierarchical view is thus flattening as more large networks interconnect directly.

Relationships between ASes:



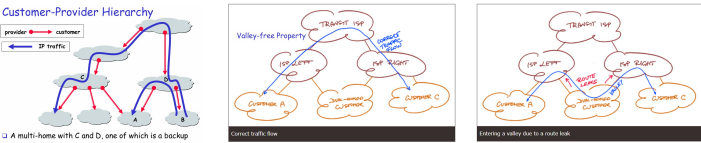
7.3 Business Relationships Between ASes

AS interconnection is primarily governed by economics and mutual agreements rather than a central authority. Two dominant business models exist:

1. **Customer-Provider Relationship:** Customer pays **provider** for Internet access and global reachability. Provider offers transit services, forwards traffic btw. customer and rest of Internet. For forwarding, traffic always flows *upstream* (from customer to provider). Customer does not carry transit traffic between providers (if multi-homed).
 - Some customers **multi-homed**, connect to multiple providers for redundancy/reliability/load balancing. They must ensure traffic from one provider not transited to other (bc. no incentive to do so).
2. **Peer-to-Peer (Settlement-Free Peering):** Two ASes may establish p2p relationship, interconnect directly to exchange traffic between their customers. Peers do not transit traffic to third parties, generally do not pay each other. *Settlement-free peering* reduces upstream transit costs and improves latency.
 - Peering policies vary: providers selectively peer based on traffic ratios, geographic presence, network scale. E.g. Verizon policy requires balanced traffic exchange (below 1.8:1 ratio), presence across multiple geographic regions, minimum interconnection capacity (e.g., OC-192 in US).

7.4 Transit, Non-Transit ASes, Valley-Free Property

- **Transit ASes** provide connectivity between their customers and other networks, often operating as ISPs.
- **Non-transit ASes**, by contrast, connect only to their own upstream providers without carrying traffic for others. Most enterprise networks, universities (e.g., NUS), and content networks fall under this category. (Multi-homed, but don't carry traffic for and between upstream providers).
- **Valley-Free Property:** A routed network with no traffic flow valleys for any source-destination inter-domain routing path pair has valley-free routing, reflecting economic hierarchy. Traffic flows **uphill** from a customer to a provider, may traverse one **peer-to-peer** link, and flows **downhill** from provider to customer. Traffic does not dip into a valley (e.g. dual-homed customer), through a route leak.
- Invalid paths such as provider to customer to peer, or peering to provider, violate business incentives and are filtered by routing policies.



7.5 Tier-1 ISPs, Internet Exchange Points (IXPs), Flattening Internet

- **Tier-1 ISPs** at top of Internet hierarchy. Reach entire Internet solely through settlement-free peering, no upstream providers. 10–12 such ASes exist worldwide (e.g., AT&T, Sprint, Level 3). Form full-mesh peering topology ensures complete reachability.
- **Internet Exchange Point (IXP)** provides shared switching fabric, enables multiple ASes to interconnect efficiently at single physical location. Promotes local traffic exchange, reduce latency and upstream transit costs.
- Many regional ISPs, content providers rely on IXPs to peer directly instead of route through upstream transit providers.
- Internet becoming increasingly **flat**. Content giants (Google, Amazon) operate private backbones, directly peer at IXPs or regional ISPs, bypass Tier-1 providers. Improves end-user latency and reduces reliance on traditional transit services.

7.6 Peering Dilemmas and Policies

- **Peering Advantages:** reduced transit costs, better performance, essential connectivity to parts of the Internet.
- **Peering Challenges:** Peers are often competitors, agreements require periodic renegotiation. Traffic imbalances or asymmetric benefits may lead to disputes.
- Consequently, peering arrangements are typically confidential and governed by private agreements or published criteria.
- **Peering Dispute & Impact:** 2005, Level 3 (AS3356) and Cogent (AS174). Level 3, a major Tier-1 provider with over 49 K on-net prefixes, accused Cogent, a fast-growing underdog, of traffic imbalance. Disconnected Cogent on 5 Oct 2005, cut off traffic between two. Around 4.3% of global prefixes became unreachable, affecting about 1% of visible ASes. High-profile victims included *The Washington Post*, *CNET Networks*, and the U.S. Department of Health and Human Services. Under public and political pressure, Level 3 restored connectivity two days later.

8. Inter-Domain Routing and Policy

8.1 Inter-Domain Routing

The Internet can be viewed as a ‘network of networks’, consisting of over 50,000 Autonomous Systems (ASes) and millions of routers supporting more than 200,000 IP prefixes.

- **Inter-domain routing:** involves connecting distinct administrative domains, each with its own policies and operational objectives.
- **Challenges for inter-domain routing protocol:**
 - **Scale:** Handling vast numbers of routers, prefixes, and ASes.
 - **Privacy:** ASes unwilling to reveal internal topology or business relationships.
 - **Policy-Driven Routing:** Each AS has own objectives e.g. minimize cost, maximize revenue, or reduce latency; no universal metric for best paths.
 - **Diverse Objectives:** Choices influence by technical & business considerations.

8.2 Intra-Domain Protocols / IGP and Limitations

Two major intra-domain routing protocols, and understand why we cannot directly borrow ideas from the protocol and where changes and relaxations are needed. Also called interior gateway protocols or IGP.

1. **Link-State Protocols:** Link-state protocols (e.g., OSPF) require every router to know the entire network topology to compute shortest paths using algorithms such as Dijkstra’s. At Internet scale, this approach is infeasible due to:
 - Flooding topology info across millions of routers too expensive. High bandwidth and storage overhead, entire path computed locally per node requires high processing overhead.
 - Privacy concerns prevent ASes from sharing internal network details.
 - No universal metric to define shortest path across heterogeneous AS policies.
2. **Distance-Vector Protocols:** Distance-vector protocols (e.g., RIP) rely on routers exchanging hop-count information with neighbors iteratively using the Bellman-Ford algorithm. Considerations for inter-domain use include:
 - Still requires a shared metric to determine the ‘best’ path.
 - Convergence is too slow for Internet-scale networks.
 - Better, hides details of network topology, only needs to determines next hop. So solution, we can extend the notion of a DV with BGP.

8.3 Border Gateway Protocol (BGP): Path-Vector (PV) Routing

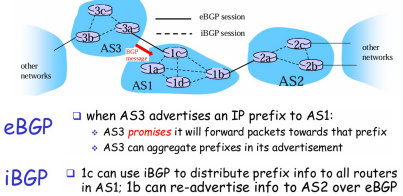
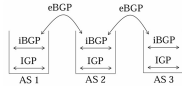
- **BGP:** the de facto inter-domain routing protocol. Extends distance-vector routing by specifying **entire AS path** instead of distance metric. Each route advertisement carries sequence of ASes to destination.
- **Advantages / Features of BGP:**
 - **Loop Prevention:** An AS can detect routing loops by checking whether its own AS number appears in the advertised path.
 - **Policy Enforcement:** AS-path information allows each AS to apply policies for route selection and advertisement.
- **Key Features of BGP**
 - Operates over TCP (port 179) to provide reliable delivery of routing information.
 - Supports policy-based decisions rather than purely metric-based selection.
 - Provides mechanisms to interconnect ASes via eBGP and propagate routes internally using iBGP.

8.4 BGP Architecture and Sessions

- **BGP Peers/Speakers:** Routers running BGP establish sessions (TCP connections) to exchange route information. There are two main types:
 - **Exterior BGP (eBGP):** Between BGP speakers in different ASes. Should be directly connected. Primarily used to exchange reachability information across AS boundaries.
 - **Internal BGP (iBGP):** Between BGP speakers within the same AS. Not required to be directly connected, IGP takes care of inter-BGP speaker connectivity. Used to propagate externally learned routes to all routers in the AS.

BGP operations

- ♦ **BGP session:** two BGP routers (or peers or speakers) exchange messages:
 - advertise *paths* to different destination network prefixes



- **iBGP Full Mesh Rule:**
 - iBGP peers must form a logical full mesh to ensure route propagation.
 - Routes learned from one iBGP peer not re-advertised to another iBGP peer to prevent loops and unnecessary flooding.
 - The underlying IGP (e.g., OSPF routing protocol) handles physical connectivity between iBGP speakers.

8.5 BGP Message Types and Mechanics

BGP maintains connectivity, route propagation w. four primary message types:

- **OPEN:** Opens TCP connection to peer, authenticates sender.
- **UPDATE:** Advertises new routes or withdraws existing ones.
- **KEEPALIVE:** Maintains the session during idle periods.
- **NOTIFICATION:** Reports errors and terminates sessions if necessary.

BGP Routes and Withdrawn Routes:

- **No expiration timer:** BGP does not use periodic timers to delete routes, unlike distance vector routing protocol RIP. Once BGP router learns routes, keep indefinitely until told no longer valid.
- **Actively withdrawn or replaced:** Routes are actively withdrawn or replaced by original advertiser through a BGP UPDATE message.
- **Invalidation:** routes from a peer become invalid when the peer goes down.

8.6 UPDATE Message Format and Route Control

BGP uses UPDATE message to advertise new routes, modify existing ones, or withdraw invalid routes. Event-driven, sends UPDATE only when routing changes occur, either withdraws prefix if unreachable, or replace existing entry with new set of path attributes and NLRI.

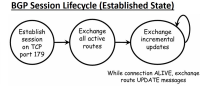
UPDATE Message Structure:

- **Withdrawn Routes:** Lists of IP prefixes that are no longer reachable. Sent actively to invalidate outdated routes.
- **Path Attributes:** Metadata describing characteristics or policies of each advertised route (e.g., AS-PATH, NEXT-HOP, LOCAL-PREF). Determines how routers select among multiple available paths.
- **Network Layer Reachability Information (NLRI):** Set of destination IP prefixes newly advertised as reachable, and their path attributes.

UPDATE Message Format

Marker (16)	
Length (2)	Type (1)
Withdrawn Routes Length (2)	Withdrawn Routes (variable)
Path Attribute Length (2)	Path Attributes (variable)
Network Layer Reachability Information (variable)	

- Withdrawn Routes: IP prefixes for the routes withdrawn
 - Can withdraw multiple routes via UPDATE message.
- Network Layer Reachability Information (NLRI): IP prefixes that can be reached from the advertised route
 - Can only advertise one feasible route for the NLRI.
 - IP prefixes are coded more compactly (refer to RFC)



8.7 Path Attributes and Policy Control

The Path Attributes field in UPDATE message is central to BGP’s flexibility and policy-based routing. Each attribute conveys information that influences route selection and propagation behavior across Autonomous Systems (ASes). BGP classifies attributes into four categories:

- **Well-Known Mandatory:** Must appear in all UPDATE messages and be recognized by all BGP routers (e.g., ORIGIN, AS-PATH, NEXT-HOP).
- **Well-Known Discretionary:** Optional to include but must be understood if present (e.g., LOCAL-PREF).
- **Optional Transitive:** May be propagated to other peers even if not recognized by a router (e.g., COMMUNITY).
- **Optional Non-Transitive:** Ignored and not propagated if not recognized (e.g., experimental or vendor-specific attributes).

Key Well-Known Mandatory Attributes

- **ORIGIN:** Indicates where the route originated—Interior Gateway Protocol (IGP), Exterior Gateway Protocol (EGP), or *incomplete* (learned indirectly).
- **AS-PATH:** Sequence of ASes traversed by the route. Used for loop prevention and as a key decision factor in path selection.
- **NEXT-HOP:** Specifies the IP address of the next-hop router outside the AS to reach advertised destination.

8.7 Forwarding Table Construction (Learning, Choosing Routes)

The router integrates BGP routes into forwarding table using following process:

1. **Route Learning:** Routes are learned via eBGP (external) and iBGP (internal). Multiple possible paths may exist for the same prefix.
2. **Best Route Selection:**
 - **Shortest AS-Path:** Prefer path with fewest AS hops.
 - **Closest NEXT-HOP:** If tied, prefer the route with the closest exit AS. (aka hot potato routing).
 - **Custom Policies:** Policies (e.g., LOCAL-PREF) can let network operators prioritize certain routes for business or performance reasons. (See 8.11)
3. **Forwarding Table Population:** Once the best path is selected, the router uses its IGP (like OSPF or IS-IS) to determine which local interface (port) should be used to forward packets to the NEXT-HOP.
4. The resulting forwarding table maps destination prefixes → output interfaces, so traffic can be sent efficiently.

8.8 BGP Routing Information Bases (RIBs)

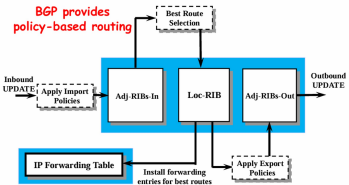
BGP keeps multiple logical tables to separate different stages of processing. Separate RIBs ensures policy control, dont advertise unapproved or invalid route.

- **Adj-RIBs-In:** Routes learned from peers, before apply import policies. Raw unprocessed routes via inbound UPDATE. (E.g. peer sent 1000 prefixes).
- **Loc-RIB:** Router’s selected best local routes after applying import policies and path finding. (E.g. Out of 3 paths to 10.0.0.0/24, install best in forwarding table).
- **Adj-RIBs-Out:** Routes to be advertised to other peers after applying export policies. (E.g. tell peer about 500 prefixes allowed by policies).

BGP Routing Information Bases

- What is a route in a BGP speaker?
 - ♦ route = prefix + attributes = NLRI + Path Attributes
- How about all the routes in a BGP speaker?
 - ♦ Routing Information Bases (RIBs)
 - ♦ RIBs = Adj-RIBs-In + Loc-RIB + Adj-RIBs-Out
 - ♦ Adj-RIBs-In: unprocessed routes from peers via inbound UPDATE; input for decision making
 - ♦ Loc-RIB: selected local routes used by the router
 - ♦ Adj-RIBs-Out: selected for advertisement to peers

BGP Decision Process: Overview

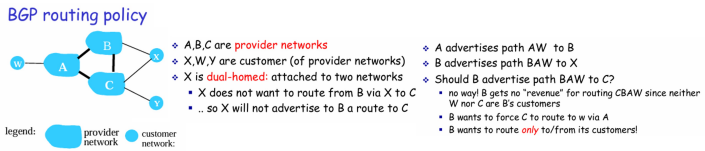


8.9 Policy Implementation in BGP (Routes to accept and announce)

- **Import Policies:** Applied to incoming routes to filter unwanted routes or rank preferred paths (e.g., using LOCAL-PREF).
- **Export Policies:** Applied to outgoing routes to enforce business relationships (See 8.10)
 - To a provider: advertise all own routes and customer routes.
 - To a customer: advertise all known routes (so customer can use).
 - To a peer: advertise only own and customer routes.
 - I.e. Never advertise peer or provider’s routes to another peer or provider.
- **AS Path Prepending:** Artificially lengthens AS-PATH to make routes less attractive, controlling inbound traffic. (E.g. don’t want traffic coming from particular provider, prepend AS number multiple times to make path longer.)

8.10 Business Relationships and BGP Policy

- **Customer-Provider Relationship:** Customer exports all own and customers’ routes to the provider. Provider exports all known routes (other customers, peers, and providers) to the customer, offering full Internet reachability.
- **Peer-to-Peer Relationship:** Peers export only own and customer routes to each other, avoid transit for routes learned from other peers or providers.
- Export/import policies as technical implementation to enforce relationships.



8.11 BGP Attributes in Route Selection

BGP uses various attributes to decide which path is preferred among multiple available routes to the same prefix. These attributes reflect both technical performance and business policy.

- **LOCAL_PREF (Local Preference):** 4-byte unsigned int (default value 100, higher = preferred), for BGP speaker to inform other internal peers of preference for a route within the same AS (used in iBGP). Use to prefer certain outbound paths (e.g., prefer customer over peer or provider). Should not send to external eBGP peers.
- **AS_PATH:** Sequence of AS numbers route traverses, prefer shorter sequences, used to prevent routing loops.
- **MULTI_EXIT_DISC (MED):** 4-byte unsigned int (default value 0, lower = more preferred), hint sent by one AS to a neighboring AS, suggest which entry point to use when multiple physical links exist between them. Influences *inbound* traffic when multiple links exist between two ASes. Share only between neighboring ASes, may be propagated over iBGP, but not propagated further to neighbouring ASes.
- **COMMUNITY:** 4-byte tag attached to routes to classify or “group” them logically. Allows bulk policy application — routers can apply import/export rules based on community tags rather than listing prefixes individually. (e.g., “no-export” community). Used to tag customer routes, control export, or trigger remote actions.

BGP best route selection

1. Calculation of degree of preference
 - If the route is learned from an internal peer, use **LOCAL_PREF** attribute or preconfigured policy
 - Otherwise, use preconfigured policy
2. Route selection (recommended process)
 - Highest degree of **LOCAL_PREF** (or the only route to the destination), and then tie breaking conditions on:
 - Smallest number of AS numbers in **AS_PATH** attribute
 - Lowest origin number in **ORIGIN** attribute
 - Most preferred **MULTI_EXIT_DISC** attribute
 - Routes from eBGP are preferred (over iBGP)
 - Lowest interior cost based on **NEXT_HOP** attribute

8.12 BGP Security Issues: Prefix Hijacking

- **Prefix Hijacking:** When an AS accidentally or maliciously advertises ownership of an IP prefix it does not own. Other ASes believe, send traffic for that prefix to it.
- **Subprefix Hijacking:** Attacker AS advertises more specific prefix (e.g., /24 of a victim’s /16), causing other AS pick that route following longest matchin prefix, and most traffic to be rerouted to the attacker.
- **Consequences:**
 - *Blackholing:* Traffic is discarded and dropped.
 - *Snooping:* Traffic is intercepted and inspected, and redirected back.
 - *Impersonation:* Traffic is redirected to fake destinations.
- **Prevention:**
 - Route filtering, where ISPs should only accept prefixes customers legitimately own. However, not everyone applies these practices, hard to filter routes initiated from far away, so BGP remains vulnerable to hijacks.
 - **RPKI (Resource Public Key Infrastructure):** Cryptographically verifies pre-fix ownership.
 - **ROA (Route Origin Authorization):** Specifies which AS is authorized to originate specific prefixes.

8.13 Summary of BGP used in Practice

Overall, BGP can be controlled in three main ways:

1. **Preference:** Modify attributes such as LOCAL_PREF or AS path prepending to prefer certain routes or providers, e.g., to reduce cost.
2. **Filtering:** Inbound/outbound filters to accept/advertise only desired routes.
3. **Tagging:** Use COMMUNITY attributes to classify routes for policy enforcement, either locally or remotely.

These mechanisms allow enforcement of business relationships, traffic engineering, and remote control (request behavior, not enforce) over how other ASes treat your routes.

9. Peer-to-Peer Networks

9.1 From Client–Server to a Decentralized Paradigm

Traditional client–server model is **asymmetric architecture**, centralized server serves multiple clients. Model faces inherent challenges:

- **Scalability:** Centralized servers become bottlenecks under high demand.
- **Single Point of Failure:** A server crash or disconnection halts service availability.

To address limitations, can use hierarchical delegation instead, distribute responsibility across multiple tiers of servers. E.g. **DNS** hierarchical structure of authoritative servers (root, TLD, domain-level). Improves load distribution, fault tolerance. In DNS, servers can respond in two distinct query modes:

- **Recursive Resolution:** The contacted server performs all subsequent lookups on behalf of the client until the final answer is obtained.
- **Iterative Resolution:** The server provides a referral (e.g., another server’s address), and the client continues querying iteratively until the answer is found.

Still, hierarchical model rely on designated servers. **Peer-to-Peer (P2P)** remove strict client–server distinction, improve scalability, robustness.

9.2 The Peer-to-Peer (P2P) Model

Peer-to-peer (P2P) system: every participating node acts as both client and server. Each can initiate requests and resources to others. **Symmetrical, decentralized architecture**, control and responsibility distributed. Key characteristics:

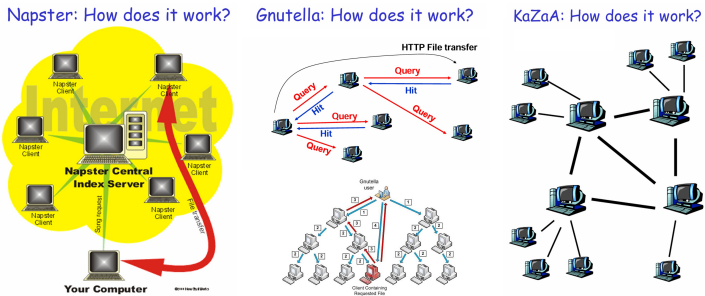
- **Self-Organization:** Dynamically peer discovery w/o central coordination.
- **Resource Sharing:** Each peer contributes bandwidth, compute etc to the network.
- **Scalability:** As more peers join, total system capacity typically increases.
- **Resilience:** The failure or departure of individual peers has limited impact since data and functions are replicated or distributed across many nodes.

Challenges:

- **Peer churn:** Frequent joining and leaving of peers disrupt stability.
- **Unreliable service:** Peers are non-dedicated and may go offline anytime.
- **Discovery:** Difficult to locate content and maintain routing in a dynamic environment. (Peers intermittently connected, change IP addresses)

9.3 Early P2P Systems: Napster, Gnutella, and KaZaA

- **Napster (Hybrid Model):**
 - Used a *centralized index server* to map files to peers.
 - Peers queried the central server to locate a file, then downloaded directly from another peer. (Used primarily for sharing MP3 files for free).
 - **Pros:** Fast, accurate keyword searches which returned info (e.g. encoding rate, file size, peer bandwidth, some info entered by user), consistent view of network.
 - **Cons:** Single point of failure of the server, large computation for queries, download from single peer only, unreliable content, legal vulnerability.
- **Gnutella (Pure P2P):**
 - Fully decentralized; peers flood the network with queries to find files.
 - **Pros:** No single authority; robust to node failure.
 - **Cons:** Inefficient due to query flooding and excessive bandwidth overhead.
- **KaZaA (Two-Tier Hierarchy):**
 - Introduced *Super Nodes (SNs)* that act as local hubs for *Ordinary Nodes (ONs)*.
 - Super Nodes form a Gnutella-like overlay among themselves.
 - **Pros:** More scalable than Gnutella; avoids Napster’s central bottleneck.
 - **Cons:** Super Nodes can become localized bottlenecks, still partially centralized.



9.4 BitTorrent: P2P for Content Distribution

BitTorrent revolutionized P2P systems by focusing on efficient **file distribution**. Each shared file forms a *swarm* of peers cooperating to exchange chunks.

Core components:

- **Tracker:** A central server maintaining a list of peers in the swarm. Server keeps track of seeds and peers in swarm but doesn't participate in actual file distribution.
- **Seed:** A peer possessing the complete file.
- **.torrent file:** Contains file metadata and tracker information.
- **torrent:** group of peers exchanging chunks of a file.
- **swarm:** seeds and peers.

How it works: A file is divided into fixed-size chunks (typically **256 KB**). The distribution process proceeds as follows:

1. A peer downloads the **.torrent** file and registers with the tracker.
2. The tracker replies with a list of active peers in the swarm.
3. The joining peer, initially having no chunks, connects to a subset of peers (its *neighbors*) and begins downloading chunks.
4. As it accumulates chunks, simultaneously uploads chunks to other peers.
5. Once a peer obtains the entire file, it may leave the swarm (*selfish*) or remain to serve others (*altruistic*).

Pulling chunks: At any time, different peers possess different subsets of chunks. Periodically, a peer queries each neighbor for list of available chunks, then requests missing chunks, prioritizing on **rarest-first**:

- Download least common chunks first, maximize data availability, robustness.

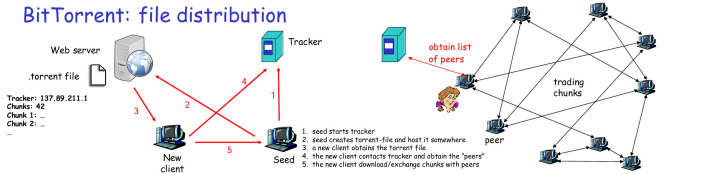
Pushing chunks: Tit-for-Tat BitTorrent policy to promote fairness, reciprocity:

- Each peer uploads chunks to a fixed number (typically four) of neighbors that currently provide it with the highest download rate.
- Every 10 seconds, the peer re-evaluates its top four upload targets.
- Every 30 seconds, it randomly selects another peer to upload to, a process known as **optimistic unchoking**. This allows discovery of new high-performing peers.

Efficiency mechanisms:

- **Rarest-first:** Prioritize downloading rare chunks to ensure availability.
- **Tit-for-tat:** Upload bandwidth is shared with peers that reciprocate; periodically unchokes random peers to find better connections.
- Dynamic reciprocity ensures that upload bandwidth is allocated fairly, prevents freeloading, and naturally balances network resources.

BitTorrent avoids Napster's legal issues as trackers do not host the actual content, only peer IPs.



9.5 Two Types of P2P Systems:

Unstructured P2P systems rely on **search-based** discovery, simple but inefficient. Structured P2P networks, introduce deterministic **addressing** using Distributed Hash Tables (DHTs), enabling efficient routing and lookup.

- **Unstructured Networks:** Peers are free to join anywhere and select neighbors arbitrarily. Objects are stored at random locations, hence need for search-based mechanisms. However, still exhibit structures like graphs or power-law topologies.
- **Structured Networks:** Enable deterministic routing and addressing. Network structure dictates where peers and objects are placed. Examples include DHT-based systems such as Chord.

9.6 Searching and Addressing:

- P2P systems can locate objects via two mechanisms:
 - **Searching:** Querying network to find object w/o knowing unique identifier.
 - **Addressing:** Using a deterministic mapping to directly locate responsible peer.
- Choice depends on how the network is formed, how and where objects are placed and the efficiency of lookup mechanisms.
- **Examples:** Google (search), DNS/IP routing (addressing), Napster / Gnutella / KaZaa / BitTorrent (mixed).

Searching vs. Addressing Comparison:

- **Addressing:** Each object is uniquely identifiable. Efficient retrieval via structured placement. Requires maintaining addressing structure and unique identifiers.
- **Searching:** More user-friendly — no need for unique names. Inefficient unless supported by strong infrastructure (e.g., Google). Requires content comparison to confirm matches.

9.7 Structured P2P and Distributed Hash Tables (DHTs)

Key Technologies: Distributed Hash Table (DHT): Core data structure enabling scalable, decentralized key-value storage. **Example architecture:** Chord.

- **Key-Value Store Abstraction:** Entries stored as (key, value) pairs. Supports operations Put(key, value), and Get(key) → returns value. Conceptually similar to a table lookup; naive lookup requires $O(N)$ time.
- **Hash Tables Recap:** Fixed-size array of hash buckets. Supports operations (insert, delete, lookup) in $O(1)$. Hash function maps keys evenly and efficiently, (e.g. $\text{hash}(x) = x \bmod 8$)
- **Distributed Hash Table (DHT):** distribute hash buckets across multiple peers. Concern is how to design, implement efficient mechanism to determine which peer responsible for which bucket, and how to route requests between peers.

DHT Principles:

- Each node is responsible for one or more hash buckets.
- As nodes join or leave, responsibility reassigns dynamically.
- Nodes communicate to discover responsible peer, scalability makes DHT efficient.
- DHTs support all hash table operations efficiently.

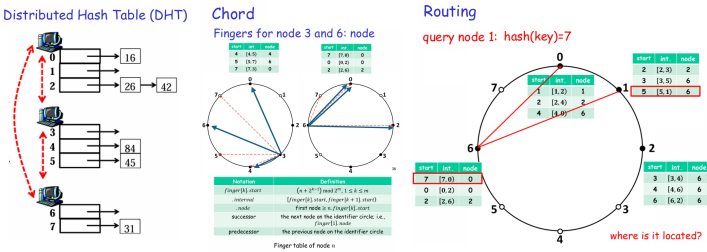
Examples of DHTs:

- **Chord (2001)** and **CAN (2001)** — canonical architectures.
- Other designs: **Pastry/Tapestry (2001)** — Plaxton-style routing. **Kademlia (2002)** — XOR metric-based routing.
- All provide the same abstraction: distributed storage and retrieval of (key, value) pairs without semantic meaning.
- Differ mainly in namespace design and routing mechanisms.

9.8 Chord: A Structured P2P Ring (DHT)

Chord is a distributed lookup protocol used in peer-to-peer (P2P) systems. Locates node storing data item in decentralized network without central directory, is a DHT implementation.

- Developed at MIT, widely used in distributed storage systems.
- Uses SHA-1 hash function to generate 160-bit identifiers for nodes and objects. Node IDs are hashed from IP addresses; object IDs from object names.
- Nodes form a logical ring with wrap-around addressing, each node maintains links to predecessor and successor.



Identifier Assignment: m -bit IDs are chosen from the range $[0, 2^m - 1]$, using hash function. Identifier space is circular. Each (key, value) pair is assigned to the node whose ID is the **successor** of the key's hash value. (key is stored at the first node whose ID $\geq$ key ID).

Naive Lookup and Finger Tables Lookup:

- **Naive:** Search for key start from any node, may require traversing up to $O(N)$ hops. (Since each node knows successor, predecessor). E.g. Looking for key 7 from node 2 → path: $2 \rightarrow 3 \rightarrow 4 \rightarrow 6 \rightarrow 0$.
- **Finger Tables:** Each node maintains a **finger table** with up to m entries (shortcuts). The i -th finger entry covers nodes at least 2^{i-1} distance away. (spaced exponentially around the ring)
- Lookup time reduces to $O(\log N)$:
 - $\text{finger}[k].\text{start} = (n + 2^{k-1}) \bmod 2^m$
 - $\text{finger}[k].\text{node} = \text{first node} \geq \text{finger}[k].\text{start}$
- **Routing Example:** Query from node 1 for hash(key) = 7. Route follows finger table shortcuts to reach the responsible node efficiently.
- **Evolution:** Chord's ring topology can generalize to multi-dimensional overlays (e.g. torus structure).

Node Join/Leave

- **Node Join** When a new node joins, it initializes its finger table and updates others'. Keys are redistributed between the new node and its successor.
- **Node Leave:** Each node maintains IPs of its two successors for redundancy. Periodically pings successors to detect failure. Upon detecting a failed peer, a node updates its successor list and informs neighbors to update finger tables to preserve routing correctness.
- Only a small number of keys require move to maintain balance. Finger tables updated incrementally. Chord can handle churn (nodes join/leave frequently).

9.9 Summary

P2P networking evolved from early unstructured and centralized hybrids like **Napster** and **Gnutella** to more efficient, structured systems such as **Chord**, **CAN**, and **BitTorrent**. Trade-off between simplicity, scalability, and efficiency. These systems are capable of decentralized storage, routing, and large-scale content distribution without centralized coordination.